ARM - OSNOVNE KARAKT.

→ RISC procesor

+ neke CISC karakt.

• 32 bit procesor

LITTLE ENDIAN

+

VON NEUMANN

* 16 registara opće namjene R0 – R15

→ aritmetičke naredbe rade s 32-bitnim podacima

* R0 – R7 → jednoznačno def.

R8 – R14 → višeznačno def.

R13 = SP stack pointer

R14 = LR link register

R15 = PC → program Counter

CPSR (Current)

SPSR (Saved)

Program States Register

REGISTRI POSEBNE NAMJENE

PC / R15 - programsko brojilo

⇒ čuva adresu sljedeće naredbe koju treba izvesti *

• nakon dohvata svake naredbe automatski se povećava za +4

↳ naredbe su u uvijek na adresama djeljivima s 4

SP / R13 - pokazivač stoga = pokazuje vrh stoga

LR / R14 - link registar = koristi se kod poziva fija za spremanje povratne adrese

Registar CPSR

N	Z	C	V	...	I	F	T	M
31	30	29	28	...	7	6	5	4 3 2 1

ARITMETIČKE ZASTAVICE

N - negative

Z - zero ⇒ true (1) ako je rezultat dao Ø → ZERO flag jer postoji

C - carry - prijenos = izlazni bit kod pomaka/rotacija

V - overflow - preljev

KONTROLNI BITOVI

I - interrupt disable (1) / enable (0)

F - fast interrupt dis (1) / enab (0)

T - thumb način rada (1) / ARM -||- (0)

M - mode - način rada ili mode procesora

Aritmetičke naredbe

naziv <sufiks> Rd, Rn, <shift_op>

ADD R0, R1, #2

Rd → R destination

Rn → R1 uvijek jedan od

shift_op
- ↳ br #
- ↳ registar
- ↳ registar s pomakom

ZASTAVICE → sufiks S iza naziva naredbe ⟹ osvježuje aritm. zastavice u CPSR

- Z i N = postavljaju ovisno o rezultatu
- C i V = ovisno o prijenosu i preljevu

Logičke naredbe: logičke operacije i rad s bitovima

Primjeri: AND R0, R0 #061111 0000 ; R0 = R0 & 1111 000

ORR R5, R6, R9 ; R5 = R6 | R9

odredišni registar

ZASTAVICE ⟹ S - osvježi ⚑ u CPSR

- Z i N = postavlja ovisno o rezultatu
- C ovisi o drugom operandu * ⟶ ako nema adresiranja s pom (Konst.)
  - → ako se koristi adresiranje s pomakom, onda se C postavlja u izlazni bit iz barrell shiftera
- V se ne mijenja

Rad s bitovima:

operacija	naredba	aktivni bit maske
postavljanje	ORR	1
Brisanje	AND	0
Brisanje	BIC	0
komplement.	EOR	1

Rad s bitovima - primjeri

Postavljanje: ORR

Maska: 0 = neutralna, 1 = postavlja bit

```
ORR   R0, R0, #0x07      ; postavlja 3 najniža bita u R0
```

Brisanje AND

Maska: 1 = neutralno, 0 = briše bit

```
AND   R0, R0, #0xFF      ; brisanje svih bajtova u R0
                           osim najnižeg
```

Brisanje: BIC

Maska 0 = neutralno, 1 = briše bit

```
BIC   R0, R0, #0xFF      ; brisanje najnižeg bajta u R0
```

Komplementiranje EOR

```
EOR   R0, R0, #0x80000000   ; komplement najvišeg bita u R0
```

NAREDBE ZA ISPITIVANJE VRIJEDNOSTI

Ispitivanje vr:

```
CMP  R0, R1                ; usporedi vrijednosti u R0 i R1
TST  R5, #0x80000000       ; ispitaj najviši bit u R5
```

→ nema odredišnog reg., ali se postavljaju zastavice u CPSR

- rez. se zanemaruje

⇒ Čuvamo stanja registara

(norm.) compare „compare negated

CMP i CMN: aritmet operacija SUB odnosno ADD

FLAG = jednako kao aritmetičke naredbe

- usporedba vrijednosti dviju operanada

„test

TST: log. operacija AND

FLAG = jednako kao logičke naredbe

- ispitivanje pojedinih bitova prvog operanda na temelju maske zadane drugim operandom

Primjeri - ispitivanje bitova

→ uspoređuje je li 0 na mjestu 1 iz #...3210 0100

Je li u bitu rednog broja 1 (drugom najmlađem) od R0 nulta?

```
TST  R0, #0b10             ; R0[1] == 0 ⟺ Z==1
```

ili isto možemo dobiti sa: ako „uvaritimo" R5

```
ANDS R5, R0, #0b10         ; obavezan sufiks „S"
```

↳ umjesto da uspoređuje, provodi log operaciju AND i ako je njen rezultat 0 onda podiže FLAG Z

TST R0, #0b111 → jesu li sva 3 najniža bita NIŠTICE!

→ Kako ispitati jesu li u 3 najniža bita R0 sve jedinice?

AND R0, R0, #0b111 ; uništava se R0

provodimo AND nad znamenke 3 od R0 i 111

rezultat se sprema u R0

CMP R0, #0b111 ; taj R0 „komadić" uspoređujemo sa 111

$R0[2....0] == 111 \Longleftrightarrow Z == 1$

npr 1001111 = R0

$$\begin{array}{r} 1001111 \\ 0000111 \\ \hline 1000111 \end{array}$$

AND

R0

uspoređujemo CMP

$100\underbrace{0111}_{} == 111$ ✗

$Z == 1$

ali zašto je FLAG zero podignut ako nema 0

PUNJENJE REGISTARA

služe za: punjenje registra vrijednošću

pomake / rotacije registra

MOV R0, #120 ; upiši u R0 broj 120

MVN R5, R6 ; upiši u R5 komplement od R6

ZASTAVICE: sufiks „S" = osvježuje CPSR

- MOV i MVN djeluju isto kao logičke naredbe

Primjer: drugi komplement broja (2k')

=> PJEŠICE: MVN R3, R3 ; $R_3 = \overline{R_3}$

ADD R3, R3, #1 $\overline{R_3} := \overline{R_3} + 1 = -R_3$

=> Brže i kraće: RSB R3, R3, #0 ; $R_3 := 0 - R_3 = -R_3$

REGISTARSKO ADRESIRANJE

ovo dosad

NEPOSREDNO ADRESIRANJE

NE KUŽIM

=> korištenje broja ↑ <imm_8r>

idealno bi bilo da ima 32 bita

ADD R1, R2, #43 *iznos broja ograničen => samo 12 bit

→ u tih 12 bitova pamti se 1 bajt koji je rotiran udesno („rotirani bajt")

- rezultat rotacije je konačna vrijednost operanda

- imamo pomični bajt proizvoljnog sadržaja

=> broj se zapisuje kao bajt rotiran udesno za paran broj bitova

- u strojnom kodu kodira s dvije skupine od 8 i 4 bita

0000 FF00 → 0000 0000 0000 0000 1111 1111 0000 0000

→ 6 × 4 = 24

0003 FC00 → 0000 0000 0000 0011 1111 1100 0000 0000

nemam pojma

NEPOSREDNO ADRESIRANJE

Rad s negativnim brojevima MVN

MVN R0, #-4

Rad s „velikim brojevima" → broj koji ima puno jedinica koje ne stanu u jedan bajt (broj koji nije u obliku rotiranog bajta)

▶ u R0 upisati 0x12340078

```
MOV  R0, #0x12000000
ORR  R0, R0, #0x00340000     ; može i ADD ili EOR
ORR  R0, R0, #0x00000078     ; može i ADD ili EOR
```

→ da smo htjeli upisati npr. -0x12340078, morali bi ga prvo prikazati u formatu 2'k kao 0xEDCBFF88 pa onda upisati u R0 sa 4 naredbe

Primjer brojeva - logičke naredbe

ORR može postaviti manji broj uzastopnih bitova (najviše 8 bit) zato što:

- jedinice u maski označavaju mjesta za postavljanje bitova
- nule u maski označavaju nepromijenjive bitove

→ postavljanje najvišeg bita u registru R0:

```
ORR R0, R0, #0x80000000
```

→ postavljanje najvišeg bajta u registru R0:

```
ORR R0, R0, #0xFF000000
```

→ „kumulativno" postavljanje dva najviša bajta u registru R0*

```
ORR R0, R0  #0xFF00 0000
ORR R0, R0  #0x00FF 0000
```

EOR može komplementirati manji broj uzastopnih bitova (najviše 8 bitova)

- jedinice u maski - mjesta za komplementiranje bitova
- nule = nepromijenjivi bitovi

nibble = niz od 4 bita

→ komplementiranje najnižeg i najvišeg nibla u reg R0

```
EOR R0, R0  #0xF000 000F
```

* komplementiranje cijelog registra treba raditi sa MVN jer je to najkraće

NAREDBE ZA MNOŽENJE PODATAKA

→ kratko i dugo množenje

kratko množenje: 32-bit rezultat

→ radi uvijek isto bez obzira na format (NBC ili 2'k)

⇒ onda kada su operandi „dovoljno mali" da stane u 32-bit

dugo množenje: 64-bit rezultat

→ radi različito za signed (2'k) i unsigned (NBC) br.

SMULL
- S → signed
- L → long

UMULL
- U → unsigned
- L → long

treba dod korekcija predznaka

⇒ rezultat uvijek unutar opsega

↳ u općenitim slučajevima: SMULL R0, R1, R5, R6 → [$R1_{High}$, $R0_{Low}$ = R5 * R6]

Primjer: R10 = 100 * R1 + R2 * R3

```
MOV  R10, #100
MUL  R10, R1, R10   →  R1·100  → R10
MUL  R4, R2, R3     →  R2·R3   → R4
ADD  R10, R10, R4   →  R10+R4  → R10
```

Množenje / dijeljenje s potencijom od 2

→ množenje s potenc. br. 2 su najbrža i ostvaruju se aritmetičkim pomakom lijevo

→ dijeljenje s potenc. br 2 → pomak desno

UVJETNO IZVOĐENJE (04)

→ sufiks

NAREDBE ZA SKOK

→ Željena adresa skoka se upiše u PC (R15 u Armu)

B - branch => običan skok → kao go to u C

BL - branch & link => poziv fje

→ skok moguć pomoću bilokoje naredbe koja mijenja PC (LDR, MOV...)
* → na svega par slučajeva, a bit trickey

- uvjetna izvođenja ili grananja (if, if-else)
- petlje (while, do while, for)

Primjer za B:

BEQ → uvjet

...

B 0x1000 → bezuvjetni skok na adresu 1000

...

ORG 0x1000 ; zadana adresa naredbe ADD

ADD R1, R2, R3 ; odredište skoka

.....

if yes

if not BEQ DALJE i uvjetni skok

SUB ; ako je lažno, sljedeća (else)

.....

TRUE!

...

DALJE ADD... ; odredište